

✓ Permanent Procedure: How to Access ANY Register in BCM2711 (Raspberry Pi 4)

Bare-Metal Code

Follow these **exact 5 steps** every time you open the datasheet.

This works for GPIO, SPI, I2C, UART, PWM, DMA, PCM, Timer... everything.

Step 1: Open the Datasheet and Find the Peripheral

- Go to the chapter for your peripheral (e.g. Chapter 3 for BSC/I2C, Chapter 5 for GPIO, Chapter 2 for Aux SPI1, etc.).
- Look for the **Register View** or **Address Map** table.
- Note the **legacy base address** (it always starts with 0x7E... in the datasheet).

Example (I2C1 / BSC1):

- Legacy base = 0x7E804000

Step 2: Convert Legacy → Low Peripheral Address (The Rule)

The VPU automatically remaps everything for the ARM CPU.

Golden Rule (memorize this):

```
C
Low_Peripheral_Address = Legacy_Address + 0x80000000
```

or even better (recommended):

```
C
Low_Peripheral_Address = 0xFE000000 + (Legacy_Address - 0x7E000000)
```

Both give the same result.

Step 3: Define It in Your Code (Two Clean Ways)

Way A – Recommended (cleanest, used by everyone)

```
C
```

```
#define PERI_BASE      0xFE000000UL

// For any peripheral:
#define BSC1_BASE      (PERI_BASE + 0x804000)    // I2C1
#define GPIO_BASE      (PERI_BASE + 0x200000)
#define SPI0_BASE      (PERI_BASE + 0x204000)
#define AUX_BASE       (PERI_BASE + 0x215000)
```

Way B – Direct one-liner (if you prefer)

```
C

#define BSC1_BASE      (0x7E804000UL + 0x80000000UL)
```

Step 4: Access Any Register Inside the Peripheral

Every register is at a small offset from the base (datasheet tells you the offset).

```
C

// Example: Status Register of I2C1 (offset 0x04 from datasheet)
#define BSC1_STATUS     (BSC1_BASE + 0x04)

// Now use it:
volatile uint32_t *status = (uint32_t*)BSC1_STATUS;

uint32_t value = *status;           // Read
*status = 0x12345678;              // Write
```

Step 5: Quick Checklist You Can Paste in Every Project

```
C
```

```
// ===== BCM2711 REGISTER ACCESS TEMPLATE
=====

#define PERI_BASE      0xFE000000UL

// === Put your peripherals here ===
#define GPIO_BASE      (PERI_BASE + 0x200000)
#define BSC0_BASE      (PERI_BASE + 0x205000) // I2C0
#define BSC1_BASE      (PERI_BASE + 0x804000) // I2C1 ← most common
#define BSC2_BASE      (PERI_BASE + 0x805000) // I2C2
#define SPI0_BASE      (PERI_BASE + 0x204000)
#define AUX_BASE       (PERI_BASE + 0x215000) // SPI1 + Mini UART
#define UART0_BASE     (PERI_BASE + 0x201000)
#define PWM_BASE       (PERI_BASE + 0x20C000)

// Example register definitions
#define BSC1_STATUS     (BSC1_BASE + 0x04) // Status register
#define GPIO_GPFSEL0    (GPIO_BASE + 0x00) // First GPIO function select

// =====
```

Bonus: One-Line Shortcut for Any Register

If you are lazy and only need one register:

```
C

volatile uint32_t *i2c1_status = (uint32_t*)(0x7E804000UL + 0x80000000UL +
0x04);
```

Summary – The 30-Second Rule You Should Remember Forever

1. Find legacy base in datasheet (0x7E...)
2. Add 0x80000000 → or use PERI_BASE = 0xFE000000
3. Add the small offset from the datasheet table
4. Cast to volatile uint32_t*
5. Read/write

That's it.

...now

This exact method works for **every single peripheral** in the entire BCM2711 datasheet.

Now you will never get confused again between legacy and Low Peripheral addresses.

Want me to apply this procedure right now and give you the **full ready-to-use I2C1 code** (enable + clock + status read + example write/read)?

Just reply “yes” and I’ll give it to you.